UNIVERSIDADE DA CIDADE DE SÃO PAULO (UNICID)
CURSO DE CIÊNCIA DA COMPUTAÇÃO
PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

Andrei Rodrigues de Barros
Gabriel Silva Medeiros
Márcia Paula de Sousa

O PROBLEMA PRODUTOR-CONSUMIDOR - CONCORRÊNCIA EM GO

Grupo 01 · Tema C1 · Paradigma Concorrente

Trabalho apresentado à disciplina de Paradigmas de Linguagens de Programação, do curso de Ciência da Computação, como requisito parcial para a avaliação A2, sob orientação do professor Cid Rodrigues de Andrade.

São Paulo
2026

Resumo

Este relatório estuda o problema clássico do produtor-consumidor com buffer limitado e a forma como a linguagem Go o resolve por meio de goroutines e canais. O trabalho está organizado em três partes. A primeira reconstrói a origem do problema, as duas condições críticas de sincronização e os riscos do modelo de memória compartilhada, apresentando a solução tradicional com semáforos, mutexes e variáveis de condição. A segunda examina o modelo *Communicating Sequential Processes* (CSP), de C. A. R. Hoare, adotado por Go, junto dos mecanismos da linguagem: goroutines, canais com buffer, semântica de bloqueio, ciclo de vida do canal e `sync.WaitGroup`. A terceira apresenta a implementação do grupo em Go, C e Java, a demonstração executável no Go Playground e a análise comparativa quanto a sintaxe, semântica, gerenciamento de memória e trade-offs. Conclui-se que o canal com buffer não acrescenta uma trava mais sofisticada ao modelo tradicional, mas desloca a responsabilidade pela sincronização do programador para a estrutura da linguagem, sem com isso eliminar todos os erros possíveis de concorrência.

Palavras-chave: concorrência; produtor-consumidor; Go; CSP; canais; sincronização.

Sumário

1 Introdução 4
2 Fundamentação teórica 5
2.1 O cenário e os três papéis 5
2.2 As duas condições críticas 5
2.3 Os perigos da memória compartilhada 6
2.4 A abordagem tradicional 7
2.5 A fragilidade do modelo tradicional 8
3 A filosofia de concorrência em Go 9
3.1 O modelo CSP 9
3.2 Da teoria ao Go 9
3.3 Goroutines e o escalonador 10
3.4 Canais com buffer 11
3.5 Ciclo de vida do canal 11
3.6 Por dentro do canal 12
3.7 Os limites do modelo 13
4 Implementação e análise comparativa 14
4.1 Arquitetura da solução 14
4.2 As duas fases da execução 15
4.3 Entrada e saída documentadas 15
4.4 Sintaxe 16
4.5 Semântica 17
4.6 Gerenciamento de memória 17
4.7 Trade-offs 18
4.8 Roteiro da demonstração 19
5 Considerações finais 19
Referências 20

1 Introdução

Programas concorrentes não se definem apenas por fazer várias coisas ao mesmo tempo. Suas partes avançam de forma independente e, por isso, precisam ser coordenadas. O problema do produtor-consumidor é o exemplo canônico dessa dificuldade: um componente gera dados, outro os consome, e os dois raramente trabalham no mesmo ritmo.

Entre eles existe um buffer de capacidade finita, e é essa finitude que cria o problema. Quando o produtor é mais rápido, o buffer enche e ele precisa esperar. Quando o consumidor é mais rápido, o buffer esvazia e é ele quem espera. Coordenar essas duas esperas, sem travar o sistema, sem desperdiçar processador e sem ler dados inválidos, constitui o problema por inteiro.

Historicamente, a solução passou pela memória compartilhada e por primitivas manuais de sincronização. Essa abordagem funciona, é ensinada em todo curso de Sistemas Operacionais e permanece em uso. Sua fragilidade, porém, é conhecida: tudo depende de o programador usar a primitiva certa, no lugar certo e na ordem certa, e uma única troca de ordem pode travar o programa em silêncio.

A linguagem Go propõe outro caminho. Em vez de proteger melhor a memória compartilhada, retira-a do modelo: as unidades de execução se coordenam trocando mensagens por canais. Este relatório investiga essa proposta, do fundamento teórico à implementação executável, e a confronta com o modelo imperativo clássico em C e com o modelo orientado a objetos em Java.

O texto segue a divisão de pesquisa adotada pelo grupo. A seção 2 trata da fundamentação teórica, a seção 3 da filosofia de concorrência em Go e a seção 4 da implementação prática e da análise comparativa.

2 Fundamentação teórica

2.1 O cenário e os três papéis

O problema envolve três elementos. O produtor gera itens de dados e os deposita em algum lugar. O consumidor retira esses itens e os processa. Entre os dois está o buffer, uma estrutura intermediária com capacidade máxima de N posições.

Convém esclarecer por que o intermediário é necessário. Caso produtor e consumidor trocassem itens diretamente, cada entrega exigiria que ambos estivessem prontos no mesmo instante, e o mais rápido ficaria parado esperando o mais lento a cada item trocado. O buffer existe para absorver essa diferença de ritmo. Enquanto houver espaço, o produtor deposita e segue trabalhando; enquanto houver itens, o consumidor retira e segue processando. Os dois só precisam se encontrar nos extremos.

Essa assimetria de velocidades não constitui defeito do sistema, e sim o comportamento esperado de partes independentes. O produtor pode ler de um disco, receber de uma rede ou calcular algum resultado, ao passo que o consumidor pode gravar, exibir ou transformar o dado recebido. Não há razão para que esses tempos coincidam.

2.2 As duas condições críticas

A capacidade finita do buffer cria duas situações que exigem tratamento.

Na primeira, o buffer está cheio. Com as N posições ocupadas, o produtor não tem onde depositar o próximo item e precisa esperar até que o consumidor libere espaço. A dificuldade não está em detectar a condição, pois basta comparar um contador com N, e sim em esperar corretamente. Uma espera implementada como laço de repetição testando a condição, prática conhecida como espera ocupada (*busy waiting*), funciona, porém consome processador sem produzir nada e prejudica justamente a thread que deveria liberar o espaço.

Na segunda situação, o buffer está vazio. Sem itens disponíveis, o consumidor não tem o que processar e precisa aguardar a chegada de um novo item. Existe aqui um risco adicional: se o consumidor não verificar a condição corretamente e tentar ler mesmo assim, lerá uma posição não inicializada, um valor nulo ou um dado antigo já consumido. O programa não necessariamente falha, apenas processa lixo e continua executando.

O problema completo pode então ser enunciado assim: esperar sem travar o sistema, sem consumir processador em espera ocupada e sem ler dado inválido.

2.3 Os perigos da memória compartilhada

No modelo tradicional, o buffer é uma estrutura na memória compartilhada, acessada por várias threads simultaneamente. Esse compartilhamento traz dois riscos clássicos, além de um terceiro problema, mais sutil, ligado à sinalização entre threads.

Uma condição de corrida (*race condition*) ocorre quando o resultado do programa depende da ordem em que as operações concorrentes são intercaladas. O exemplo mais direto está no contador de itens do buffer. Incrementar um contador parece uma operação única, mas no nível da máquina são três passos: ler o valor da memória para um registrador, somar um e escrever o resultado de volta. Se o produtor executa `count++` enquanto o consumidor executa `count--`, sem proteção, os passos podem se intercalar conforme a tabela a seguir.

Passo	Produtor (`count++`)	Consumidor (`count--`)	`count` na memória
A1	lê count, reg = 5		5
B1		lê count, reg = 5	5
A2	reg + 1, escreve 6		6
B2		reg − 1, escreve 4	4

Um incremento seguido de um decremento deveria manter o contador em 5. A intercalação acima o deixa em 4, e outra ordem o deixaria em 6. O contador passa a mentir sobre o conteúdo real do buffer, enquanto o programa segue executando como se nada tivesse acontecido. O mesmo raciocínio vale para os índices de entrada e saída do buffer circular.

O segundo risco é o impasse (*deadlock*), situação em que os participantes ficam bloqueados indefinidamente, cada um esperando uma condição que só o outro poderia satisfazer e que, por estar igualmente bloqueado, nunca satisfará. Importa distinguir impasse de espera comum. Esperar por espaço num buffer cheio é saudável, pois em algum momento o consumidor retira um item e libera a vaga. No impasse, a condição aguardada jamais ocorrerá. A característica mais perigosa desse

cenário é o silêncio: em C, o programa não emite erro nem trava a máquina, simplesmente para de progredir.

Tanenbaum e Bos (2016) recorrem a um cenário específico para mostrar por que soluções ingênuas de sinalização falham. Considere-se uma implementação com as primitivas `sleep` e `wakeup`. O consumidor verifica o contador, encontra zero e decide dormir. Antes que ele efetivamente durma, porém, o escalonador troca para o produtor, que insere um item, faz o contador passar a um e envia um `wakeup`. Como o consumidor ainda não está dormindo, o sinal se perde, já que `wakeup` não é armazenado em lugar nenhum. O consumidor então dorme, para sempre, e o produtor, ao encher o buffer, dorme também. A lição que se extrai do episódio é que a verificação da condição e a decisão de dormir precisam ser atômicas, ou então o sinal precisa ser guardado para uso futuro. Dessa segunda ideia nasce o semáforo.

2.4 A abordagem tradicional

A solução clássica tem duas variantes: semáforos, como em Tanenbaum e Bos (2016), ou mutex com variáveis de condição, como em Pthreads. A primeira usa semáforos contadores. Um semáforo é, conceitualmente, um contador de recursos com duas operações atômicas: `down`, que decrementa o valor ou bloqueia a thread caso ele já seja zero, e `up`, que incrementa e acorda uma thread porventura bloqueada. A solução emprega dois semáforos desse tipo: `empty`, inicializado com N, conta os espaços livres, e `full`, inicializado com zero, conta os itens disponíveis. Como o valor fica armazenado no contador, o problema do sinal perdido desaparece.

Completa essa variante o semáforo binário, ou mutex, inicializado em um, que garante a exclusão mútua e assegura que apenas uma thread por vez manipule o buffer e seus índices. O esqueleto da solução assume então a forma abaixo.

```
void produtor() {
    while (1) {
        item = produzir();
        down(&empty);
        down(&mutex);
        inserir(item);
        up(&mutex);
        up(&full);
    }
}
```

```
void consumidor() {
    while (1) {
        down(&full);
        down(&mutex);
        item = remover();
        up(&mutex);
        up(&empty);
        consumir(item);
    }
}
```

A segunda variante usa variáveis de condição. Em Pthreads, a mesma coordenação costuma ser escrita com um `pthread_mutex_t` e duas variáveis de condição. A operação `pthread_cond_wait` libera o mutex e suspende a thread de forma atômica, resolvendo por outro caminho o problema do sinal perdido.

```
pthread_mutex_lock(&trava);
while (tam == CAPACIDADE) {
    pthread_cond_wait(&tem_espaco, &trava);
}
/* insere o item e atualiza os índices */
pthread_cond_signal(&tem_item);
pthread_mutex_unlock(&trava);
```

Dois detalhes desse trecho merecem atenção, por serem origem frequente de erro. O primeiro é o uso de `while`, e não de `if`, no teste da condição. Entre o momento em que a thread é acordada e o momento em que volta a executar, outra thread pode ter ocupado a vaga; com `if`, a thread seguiria adiante supondo que a condição ainda vale. O segundo detalhe está na escolha entre `signal` e `broadcast`. Durante a operação normal, acordar uma única thread é suficiente. Ao encerrar a produção, no entanto, todos os consumidores precisam acordar para descobrir que não virão mais itens, o que exige `pthread_cond_broadcast`. Em Java, o mesmo padrão aparece sob a forma de monitor: a trava fica embutida no próprio objeto, `synchronized` garante uma thread por vez e os métodos `wait()` e `notifyAll()` fazem a coordenação.

2.5 A fragilidade do modelo tradicional

A solução descrita é correta, mas depende inteiramente da disciplina de quem a escreve. O exemplo mais eloquente dessa fragilidade está na ordem das operações de sincronização. Considere-se a versão do produtor com apenas duas linhas trocadas de lugar:

```
down(&mutex);     /* adquire a trava antes de reservar espaço */
down(&empty);     /* e só então verifica se há espaço */
```

Caso o buffer esteja cheio, o produtor adquire o mutex e em seguida adormece esperando espaço, segurando a trava. O consumidor, que seria quem liberaria esse espaço, precisa do mutex para retirar um item e fica bloqueado. Nenhum dos dois acorda.

As duas versões usam exatamente as mesmas chamadas, e a diferença está apenas na ordem. O compilador não emite aviso algum, e o erro se manifesta somente em execução, por vezes de maneira intermitente, conforme o escalonamento.

É esse conjunto de armadilhas, que vai do contador corrompido por intercalação ao sinal perdido, do `if` no lugar do `while` ao `signal` no lugar do `broadcast`, até a ordem invertida de travas, que motiva a busca por um modelo no qual a sincronização não dependa da atenção do programador a cada linha escrita.

3 A filosofia de concorrência em Go

3.1 O modelo CSP

Em 1978, o cientista da computação britânico C. A. R. Hoare publicou na *Communications of the ACM* o artigo *Communicating Sequential Processes*. Numa época em que a memória compartilhada já era o mecanismo usual para explorar multiprocessadores, Hoare propôs uma álgebra de processos na qual o compartilhamento de memória é simplesmente banido do modelo.

O CSP apoia-se em três ideias. Processos são unidades de execução isoladas, e nenhum deles compartilha estado interno com outro. A sequencialidade garante que, dentro de cada processo, as instruções sigam uma ordem única, sem concorrência interna. A comunicação ocorre por canais de mensagens, nos quais o envio e o recebimento constituem um evento acoplado. No modelo original, o processo que envia bloqueia até que o destinatário esteja pronto para receber, no que se convencionou chamar de *rendezvous*, de modo que a passagem de dados e o controle de fluxo se tornam a mesma coisa.

A diferença em relação ao modelo tradicional é de natureza, e não de grau. Não se trata de proteger melhor a memória compartilhada, e sim de eliminá-la do modelo mental com que o programa é concebido.

3.2 Da teoria ao Go

A passagem do CSP de formalismo matemático para construção nativa de uma linguagem de uso corrente levou décadas e ocorreu em grande parte no Bell Labs. Na década de 1980, Luca Cardelli e Rob Pike desenvolveram a linguagem Squeak, aplicando o CSP à comunicação com dispositivos de entrada em interfaces gráficas. Em 1989 veio Newsqueak, que tratou canais como objetos de primeira classe e permitiu criar processos e canais dinamicamente em tempo de execução. Nos anos 1990 seguiram-se as linguagens Alef e Limbo. Em 2007, no Google, Robert Griesemer, Rob Pike e Ken Thompson iniciaram o projeto de Go, anunciado publicamente em 2009, que uniu a sintaxe da família C a essa herança de concorrência baseada em CSP.

A filosofia da linguagem é resumida em uma frase: “Não comunique compartilhando memória; em vez disso, compartilhe memória comunicando”. A frase consta do documento *Effective Go*, da documentação oficial, e foi divulgada no artigo *Share Memory By Communicating*, publicado por Andrew Gerrand no blog oficial de Go em 2010; depois, foi retomada por Rob Pike entre os *Go Proverbs*, em 2015. Parte

do material consultado durante a pesquisa atribuía o lema apenas a Rob Pike; como o *Effective Go* é a fonte escrita mais antiga localizada, o relatório o cita como fonte, em vez de atribuir a frase a uma pessoa.

O conteúdo do lema é a transferência de propriedade. Em lugar de duas unidades de execução acessarem o mesmo dado sob proteção de uma trava, o dado viaja pelo canal: quem enviou deixa de usá-lo e quem recebeu passa a ser responsável por ele. Registre-se, porém, que essa transferência é uma disciplina que o canal facilita, e não uma proibição imposta pelo compilador. Nada impede que um programa envie um ponteiro por um canal e continue alterando o dado apontado, caso em que a memória volta a ser compartilhada, com todos os riscos decorrentes.

3.3 Goroutines e o escalonador

Uma goroutine é a unidade de execução concorrente de Go, criada com a palavra-chave `go`, como em `go produtor(canal)`. Ela não é uma thread do sistema operacional, mas uma unidade gerenciada pelo *runtime* da linguagem no espaço de usuário. Na implementação atual, a pilha de uma goroutine tem no mínimo dois kilobytes, valor definido pela constante `stackMin = 2048` em `runtime/stack.go`; desde o Go 1.19, pode começar maior, segundo a média de uso das goroutines anteriores, e cresce e encolhe conforme a necessidade, com o runtime realocando e ajustando os ponteiros.

Uma thread do sistema operacional, usada por C através de Pthreads e pelas threads de plataforma de Java, reserva uma pilha consideravelmente maior, cujo tamanho é definido pelo sistema e pode ser configurado. Além do consumo de memória, a troca de contexto entre threads do sistema passa pelo núcleo, ao passo que a comutação entre goroutines ocorre no espaço de usuário e envolve salvar poucos registradores. O material de pesquisa inicial afirmava que uma thread do sistema ocupa de um a dois megabytes; como esse valor depende do sistema operacional e da configuração, e nenhuma das fontes consultadas o confirma, ele foi retirado, mantendo-se apenas o valor da goroutine, verificável no código-fonte do runtime.

O runtime distribui as goroutines sobre as threads do sistema por meio de três abstrações, conhecidas pelas iniciais G, M e P. A letra G designa a goroutine, com sua pilha e seu estado. A letra M designa a thread do sistema operacional. A letra P designa o contexto lógico que mantém uma fila local de goroutines prontas, cuja quantidade é definida pela variável `GOMAXPROCS`. O escalonador emprega a técnica de

work-stealing: quando um P esvazia a própria fila, toma parte das goroutines da fila de outro P, mantendo os núcleos ocupados. Quando uma goroutine realiza uma chamada de sistema bloqueante, o runtime pode desacoplar a thread bloqueada e associar o contexto P às demais goroutines em outra thread, evitando que todo o conjunto pare.

3.4 Canais com buffer

Um canal é o conduto tipado pelo qual as goroutines se comunicam. A instrução `make(chan int, 3)` cria um canal que transporta inteiros e tem capacidade para três valores, enquanto as funções `len(canal)` e `cap(canal)` informam, respectivamente, quantos itens estão no buffer e qual é a capacidade total.

A correspondência com o problema clássico é direta. O vetor de N posições torna-se a capacidade do canal, o contador de itens torna-se `len(canal)`, os índices de entrada e saída passam a ser mantidos pelo runtime, o mutex torna-se uma trava interna e os semáforos `empty` e `full` são substituídos pelo bloqueio embutido nas operações de envio e recebimento. Tudo o que o programador coordenava manualmente passa a ser propriedade de uma única estrutura.

A semântica de bloqueio é a propriedade central do modelo, e vale enunciá-la com precisão: o bloqueio não é algo que o programador escreve, mas o comportamento das próprias operações. No envio, escrito `canal <- item`, se o canal estiver cheio a goroutine que envia é suspensa naquela linha; quando outra goroutine retira um item e abre espaço, o envio prossegue sozinho. No recebimento, escrito `item := <-canal`, se o canal estiver vazio a goroutine que recebe é suspensa, voltando a executar assim que um valor é enviado.

As duas condições críticas identificadas na seção 2.2 estão, portanto, resolvidas pela semântica da linguagem. Não há semáforo a declarar, variável de condição a sinalizar, flag a verificar nem laço de espera ocupada a escrever. Num canal sem buffer, criado com `make(chan int)`, cada envio só se completa quando há um receptor pronto, o que reproduz o *rendezvous* do CSP original em sua forma mais fiel.

3.5 Ciclo de vida do canal

Quando o produtor termina, sinaliza o fim da transmissão com `close(canal)`. Do lado do consumidor, o laço idiomático é `for item := range canal`, cujo comportamento merece ser descrito com cuidado, pois ele não encerra no instante do fechamento. O laço continua entregando os itens que ainda estão no buffer e só

termina quando o canal está fechado e vazio, de modo que nenhum item se perde quando o produtor fecha o canal ainda cheio.

Duas regras acompanham o fechamento. Enviar para um canal fechado provoca `panic`, assim como fechar o mesmo canal duas vezes. Por isso a convenção estabelece que quem fecha o canal é o produtor, ou seja, quem envia.

Go permite ainda restringir a direção do canal na assinatura da função. Com `chan<- int`, a seta aponta para dentro do canal e a função só pode enviar; com `<- chan int`, a seta sai e a função só pode receber. Caso o consumidor tente enviar, o programa não compila. O ganho é duplo: os papéis do problema ficam documentados no próprio tipo e a verificação acontece em tempo de compilação, e não como erro que aparece em execução, em alguma intercalação infeliz. Trata-se do princípio do menor privilégio aplicado às assinaturas de função.

Canais transportam dados, mas em muitos casos é preciso apenas saber quando todas as goroutines terminaram. Para isso existe o `sync.WaitGroup`, um contador operado por três funções: `Add` registra o trabalho iniciado, `Done` sinaliza a conclusão de uma goroutine, geralmente sob um `defer`, e `Wait` bloqueia até o contador chegar a zero. Sem esse mecanismo, a função `main` poderia terminar antes de os consumidores processarem os últimos itens, e o encerramento da `main` encerra o programa inteiro. Além da contagem, o `WaitGroup` participa do modelo de memória de Go, pois as chamadas a `Done` ocorrem antes do retorno de `Wait`, o que garante que tudo o que as goroutines escreveram esteja visível depois que `Wait` retorna, sem necessidade de barreiras de memória manuais.

3.6 Por dentro do canal

Esta seção descreve a implementação atual do runtime, documentada em `src/runtime/chan.go`. Não se trata de comportamento garantido pela especificação da linguagem, e os detalhes podem mudar entre versões.

Um canal com buffer é uma estrutura alocada no heap, chamada `hchan`. Entre seus campos estão `qcount`, que guarda a quantidade de elementos atualmente no buffer, e `dataqsiz`, que guarda a capacidade informada em `make`. O campo `buf` aponta para o array circular, enquanto `elemsize` e `elemtype` determinam o tamanho e o tipo do elemento, permitindo a cópia correta. Os índices `sendx` e `recvx` controlam a próxima escrita e a próxima leitura, e as filas `sendq` e `recvq` guardam as goroutines

bloqueadas para enviar ou receber. Há ainda o campo `closed`, que marca se o canal foi fechado, e o campo `lock`.

Esse último campo é o detalhe conceitualmente mais interessante, pois revela que o canal usa uma trava internamente. A diferença em relação ao modelo tradicional não está, portanto, na ausência de travas, e sim no fato de essa trava residir no runtime, fora do alcance e da responsabilidade de quem escreve o programa. Os índices `sendx` e `recvx`, por sua vez, voltam ao início ao chegar ao fim do array, fazendo do buffer uma fila circular, exatamente a mesma estrutura que em C o programador implementa com a expressão `(indice + 1) % CAPACIDADE`.

Quando uma operação não pode prosseguir, o runtime registra a goroutine em `sendq` ou `recvq` por meio de uma estrutura chamada `sudog`, que guarda um ponteiro para a goroutine suspensa e o endereço de onde o dado deve ser lido ou escrito. A goroutine é então estacionada pela função `gopark` e sai do processador, sem consumir CPU enquanto espera. Quando outra goroutine satisfaz a condição, o dado é copiado e a goroutine estacionada volta à fila de execução por meio de `goready`. Trata-se, em essência, da mesma espera que em C se escreve com semáforos e variáveis de condição, agora executada pelo runtime.

3.7 Os limites do modelo

A literatura de divulgação frequentemente afirma que os canais eliminam condições de corrida e impasses por projeto, no que se costuma chamar de *safety by design*. A afirmação é imprecisa, e o grupo registra aqui os limites reais do modelo, apurados durante a pesquisa.

Condições de corrida continuam possíveis. O canal protege o dado que é transferido por ele, mas se o programa enviar um ponteiro e continuar alterando o dado apontado, há memória compartilhada outra vez. O detector acionado por `go run -race` existe precisamente porque corridas podem ocorrer em Go.

Impasses também continuam possíveis. Esquecer o `close`, ou criar uma dependência circular entre goroutines, produz bloqueio permanente. A diferença em relação a C é diagnóstica, e não trivial: quando todas as goroutines estão bloqueadas, o runtime detecta a situação e aborta com a mensagem `fatal error: all goroutines are asleep - deadlock!`, ao passo que em C o programa simplesmente congela.

Por fim, canais não substituem todas as primitivas. Para proteger um estado simples, como um contador, um `sync.Mutex` ou o pacote `sync/atomic` costumam ser mais claros e mais eficientes do que passar mensagens, e a própria biblioteca padrão de Go oferece essas alternativas.

A formulação precisa, portanto, é a seguinte: o canal resolve o caso do produtor-consumidor com segurança estrutural, deslocando a sincronização do programador para a linguagem, mas não torna impossível escrever programas concorrentes incorretos. O tópico de pesquisa original previa discutir como os canais previnem corridas de dados por projeto; a investigação mostrou que essa formulação é mais forte do que o modelo sustenta, e o texto acima apresenta a versão verificada.

4 Implementação e análise comparativa

4.1 Arquitetura da solução

O grupo implementou o produtor-consumidor em Go e, para efeito de comparação, o mesmo programa em C e em Java, com o mesmo comportamento e saídas equivalentes. Os parâmetros são comuns às três versões: buffer de capacidade três, dez itens, dois consumidores e tempo de processamento de 300 milissegundos por item.

A função produtora gera os itens, simula o tempo de produção com `time.Sleep` e envia cada item ao canal, fechando-o ao terminar.

```
func produtor(canal chan<- int) {
    for item := 1; item <= totalItens; item++ {
        time.Sleep(tempoDeProducao(item))
        canal <- item          // espera aqui enquanto não houver
espaço
        registrar(...)
    }
    close(canal)               // avisa que não virão mais itens
}
```

A função consumidora recebe do canal com `range` e simula o processamento de cada item.

```
func consumidor(id int, canal <-chan int, grupo *sync.WaitGroup) {
    defer grupo.Done()
    for item := range canal {
        registrar(...)
        time.Sleep(tempoConsumo)
    }
}
```

A função principal cria o canal, dispara as goroutines e aguarda o encerramento dos consumidores.

```
func main() {
    canal := make(chan int, capacidade)
    var grupo sync.WaitGroup

    go produtor(canal)                        // o produtor fecha o canal

    for id := 1; id <= consumidores; id++ {
        grupo.Add(1)
        go consumidor(id, canal, &grupo)
    }
    grupo.Wait()
}
```

Não é necessário esperar pelo produtor separadamente, pois ele fecha o canal ao terminar e os consumidores só encerram depois que todo o trabalho acabou.

4.2 As duas fases da execução

Um primeiro teste revelou um problema de demonstração. Com produção e consumo equilibrados, o buffer nunca enchia e nenhuma das duas condições críticas aparecia na saída, de modo que o programa funcionava, mas não mostrava o problema que o trabalho se propõe a explicar.

A execução foi então estruturada em duas fases. Na primeira, a produção é lenta, com 350 milissegundos por item: o consumo supera a produção, o canal vive vazio e são os consumidores que esperam. Na segunda, ocorre uma rajada de produção, com 20 milissegundos por item: a produção supera o consumo, o canal enche até três de três posições e passa a ser o produtor quem espera. Uma única execução exibe, assim, as duas condições.

4.3 Entrada e saída documentadas

O programa não lê do teclado. A entrada são as constantes declaradas no início do arquivo.

```
const (
    capacidade   = 3  // tamanho do buffer do canal
    totalItens   = 10 // itens a produzir
    itensLentos  = 4  // itens da fase 1
    consumidores = 2  // goroutines consumidoras

    tempoLento   = 350 * time.Millisecond
    tempoRapido  = 20 * time.Millisecond
    tempoConsumo = 300 * time.Millisecond
)
```

Reproduz-se a seguir um trecho da saída real, com os dois momentos decisivos.

```
--- FASE 1: produção lenta, os consumidores vão esperar ---
[ 0.35s] Produtor     | canal 0/3 | produziu o item 1
[ 0.35s] Consumidor 2 | canal 0/3 | consumiu o item 1
```

```
[ 0.65s] Consumidor 2 | canal 0/3 | canal vazio, esperando item

--- FASE 2: rajada de produção, o canal vai encher ---
[ 1.46s] Produtor     | canal 2/3 | produziu o item 7
[ 1.48s] Produtor     | canal 3/3 | produziu o item 8
[ 1.50s] Produtor     | canal 3/3 | canal cheio, esperando espaço
[ 1.70s] Consumidor 1 | canal 3/3 | consumiu o item 6
[ 1.70s] Produtor     | canal 3/3 | produziu o item 9
[ 2.61s] Consumidor 1 | canal 0/3 | canal fechado e vazio, encerrou

Os 10 itens foram produzidos e consumidos.
```

Por se tratar de programa concorrente, a ordem exata das linhas e a identidade do consumidor que recebe cada item variam entre execuções. Permanecem válidas, em qualquer execução, as seguintes propriedades: cada item é enviado uma vez e consumido uma vez, nada se perde nem se repete, e o canal nunca ultrapassa três posições.

4.4 Sintaxe

Contando apenas linhas de código, sem linhas em branco e sem comentários, os três programas, que fazem exatamente a mesma coisa, apresentam a seguinte

Linguagem	Linhas de código	O que o programador manipula
Go	67	`chan`, `go`, `close`, `range`, `sync.WaitGroup`
Java	134	vetor circular, índices, `synchronized`, `wait()`, `notifyAll()`, flag de término
C	173	vetor circular, índices, mutex, duas variáveis de condição, flag, `malloc` e `free`

extensão.

O envio de um item sintetiza a diferença entre as abordagens. Em Go, a operação se resume a `canal <- item`. Em C, a mesma operação exige trancar, testar a condição em laço, escrever no buffer, atualizar os índices, sinalizar e destrancar.

```
pthread_mutex_lock(&buffer->trava);
while (buffer->tam == CAPACIDADE) {
    pthread_cond_wait(&buffer->tem_espaco, &buffer->trava);
}
buffer->itens[buffer->fim] = item;
buffer->fim = (buffer->fim + 1) % CAPACIDADE;
buffer->tam++;
pthread_cond_signal(&buffer->tem_item);
pthread_mutex_unlock(&buffer->trava);
```

No consumidor, a construção `for item := range canal` substitui todo o protocolo de término que C e Java precisam montar manualmente, incluindo a flag de encerramento, o `broadcast` final e o teste que distingue a situação de buffer vazio no momento daquela de buffer vazio em definitivo. Cabe registrar que parte da diferença de linhas vem do registro formatado da saída, mais trabalhoso em C; o núcleo do problema, composto por buffer, espera e término, é o que mais encolhe.

4.5 Semântica

Em C e Java, as threads compartilham a mesma estrutura, e a correção depende de todas passarem pela trava. Em Go, o dado é transferido pelo canal, o que elimina o acesso simultâneo à mesma posição de memória.

O bloqueio, em Go, é propriedade da operação. Em C e Java, precisa ser reconstruído com `while` e `wait`, e os erros clássicos que daí decorrem, como trocar `while` por `if` ou inverter a ordem das travas, estão sempre à espera do programador desatento.

As três linguagens são estática e fortemente tipadas. A diferença reside em que, em Go, o canal é um tipo de primeira classe e carrega a direção: `chan<- int` e `<-chan int` são verificados pelo compilador, ao passo que em C e Java nada impede, no nível do tipo, que uma thread consumidora escreva no buffer.

Quanto ao ciclo de vida, `close` é uma operação da linguagem, com semântica definida: o `range` drena e termina, e enviar para canal fechado gera `panic`, ou seja, um erro explícito em lugar de corrupção silenciosa.

O escopo é léxico, com blocos e closures, e a avaliação é estrita, como em C e Java. A diferença relevante está no modelo de memória, pois o *Go Memory Model* garante que o envio no canal ocorre antes do recebimento correspondente, de modo que quem recebe enxerga tudo o que o produtor escreveu antes de enviar, sem necessidade de `volatile` ou de barreiras manuais. Registre-se ainda a diferença diagnóstica já mencionada: quando todas as goroutines ficam bloqueadas, o runtime aborta com mensagem explícita, enquanto em C o programa congela sem aviso.

4.6 Gerenciamento de memória

Em C, a versão do grupo aloca a estrutura do buffer e o vetor de itens com `malloc`, tratando a eventual falta de memória, e libera tudo em `destruir_buffer()`. A ordem importa: o vetor é liberado antes da estrutura que o contém, pois liberar a estrutura primeiro perderia o endereço do vetor. A limpeza vai além da memória, já

que o mutex e as duas variáveis de condição também precisam ser destruídos. A verificação com Valgrind confirmou a ausência de vazamentos, registrando 42 alocações e 42 liberações, com a mensagem `All heap blocks were freed -- no leaks are possible`.

Em Java, todo objeto é alocado dinamicamente no heap, de modo que cada `new` é uma alocação. O que não existe é a liberação manual: quando nenhuma referência alcança mais o objeto, o coletor de lixo o recolhe.

Em Go, o coletor de lixo é concorrente e o compilador realiza análise de escape para decidir se cada valor fica na pilha ou no heap. O canal é alocado no heap por `make`, e os itens enviados são copiados para o buffer interno, de maneira que produtor e consumidor não escrevem na mesma posição ao mesmo tempo. Nenhuma linha do programa libera memória.

A tabela a seguir resume as três abordagens.

	C	Java	Go
Como aloca	`malloc`, explícito no código	`new`, todo objeto no heap	`make` e `new`
Quem libera	o programador, com `free`	o coletor de lixo	o coletor de lixo
Riscos	vazamento, uso após liberação, liberação dupla	—	—
Outros recursos	mutex e variáveis de condição destruídos manualmente	o monitor desaparece com o objeto	o canal desaparece com o coletor

4.7 Trade-offs

A favor do modelo de Go, para este problema específico, pesa o fato de a exclusão mútua deixar de ser disciplina do programador e passar a ser propriedade da estrutura, pois não existe sequência de envios e recebimentos capaz de deixar o buffer inconsistente. Soma-se a isso a redução de código e de pontos de falha, visto que não há como esquecer um `unlock`, trocar `while` por `if` ou inverter a ordem de duas travas. O término também é mais limpo, apoiado em `close` e `range`, com a direção do canal verificada em tempo de compilação.

Entre as limitações, estão aquelas já discutidas na seção 3.7, às quais se acrescenta o fato de o canal esconder uma trava interna, o que o torna menos indicado para estado compartilhado simples, onde `sync.Mutex` pode ser mais claro e mais rápido. Há igualmente menos controle de baixo nível do que em C, uma vez que escalonamento e coleta de lixo ficam a cargo do runtime, o que pode pesar em sistemas de tempo real ou com memória restrita.

O paradigma imperativo com threads, representado por C, oferece controle total sobre memória e escalonamento, ao custo de uma sincronização manual e propensa a erros. O paradigma orientado a objetos, representado por Java, organiza o estado em objetos e embute a trava no monitor, mas permanece baseado em memória compartilhada, ainda que bibliotecas como `java.util.concurrent.BlockingQueue` aproximem o modelo daquele adotado por Go.

4.8 Roteiro da demonstração

A demonstração ocorre no Go Playground, ambiente homologado para o tema, por meio de permalink. O roteiro prevê uma execução inicial sem alterações, na qual a fase 1 mostra os consumidores esperando e a fase 2 mostra o canal enchendo e o produtor esperando. Seguem-se três variações, com previsão solicitada ao público antes de cada uma. Reduzindo a capacidade para um, o produtor passa a esperar a cada item da rajada. Com um único consumidor, o produtor espera várias vezes na rajada. Comentando a linha `close(canal)` e o log abaixo dela, que sozinho anunciaria um falso fechamento, o laço `range` nunca termina e o runtime aborta com o erro de impasse.

Cabe uma observação metodológica. O Go Playground utiliza relógio simulado e determinístico, de modo que os tempos exibidos indicam ordem e bloqueio, e não desempenho real. Nenhuma medição de velocidade deve ser inferida da saída do programa.

5 Considerações finais

O problema do produtor-consumidor é, antes de tudo, um problema de coordenação. As duas condições críticas, buffer cheio e buffer vazio, não desaparecem em nenhum modelo de concorrência. O que muda é onde reside a responsabilidade por tratá-las.

No modelo imperativo clássico, essa responsabilidade cabe ao programador a cada linha escrita: declarar os semáforos corretos, adquiri-los na ordem certa, testar a

condição em laço, escolher entre sinalizar um ou todos. O modelo funciona e oferece controle fino, mas é frágil, e sua fragilidade não está em erros grosseiros, e sim em duas linhas trocadas de lugar.

Em Go, a mesma coordenação é propriedade do canal. O buffer limitado é a capacidade do canal, a espera por espaço e por item é a semântica do envio e do recebimento, o término se dá por `close` com `range`, e os papéis de produtor e consumidor são verificados pelo compilador. O programa resultante tem 67 linhas, contra 173 da versão em C, e os erros clássicos do modelo tradicional não encontram onde ocorrer.

Essa conclusão, contudo, precisa ser enunciada com precisão. Canais não tornam a concorrência segura por si sós, pois impasses continuam possíveis, corridas continuam possíveis quando se compartilha memória por outras vias, e o canal possui internamente a mesma trava que o modelo tradicional expõe. O ganho real é mais modesto, e mais útil, do que a versão publicitária costuma sugerir: para a classe de problemas que o canal foi concebido para resolver, o caminho correto é também o caminho mais curto, e é isso, na prática, o que mais reduz a incidência de erros.

Referências

COX, Russ. **Bell Labs and CSP threads**. Disponível em: https://swtch.com/~rsc/thread/ Acesso em: 1 out. 2026.

GERRAND, Andrew. **Share memory by communicating**. The Go Blog, 13 jul. 2010. Disponível em: https://go.dev/blog/codelab-share Acesso em: 3 out. 2026.

HOARE, Charles Antony Richard. Communicating sequential processes. **Communications of the ACM**, New York, v. 21, n. 8, p. 666-677, ago. 1978.

PIKE, Rob. **Go Proverbs**. Gopherfest SV, 2015. Disponível em: https://go-proverbs.github.io/ Acesso em: 13 out. 2026.

TANENBAUM, Andrew S.; BOS, Herbert. **Sistemas operacionais modernos**. 4. ed. São Paulo: Pearson, 2016.

THE GO AUTHORS. **Effective Go**. Disponível em: https://go.dev/doc/effective_go Acesso em: 3 out. 2026.

THE GO AUTHORS. **Go 1.19 release notes**. Disponível em: https://go.dev/doc/go1.19 Acesso em: 13 out. 2026.

THE GO AUTHORS. **The Go memory model**. Disponível em: https://go.dev/ref/mem Acesso em: 1 out. 2026.

THE GO AUTHORS. **The Go programming language specification**. Disponível em: https://go.dev/ref/spec Acesso em: 3 out. 2026.

THE GO AUTHORS. **runtime/chan.go**: implementação de canais. Disponível em: https://github.com/golang/go/blob/master/src/runtime/chan.go Acesso em: 1 out. 2026.

THE GO AUTHORS. **runtime/stack.go**: constante stackMin. Disponível em: https://github.com/golang/go/blob/master/src/runtime/stack.go Acesso em: 3 out. 2026.